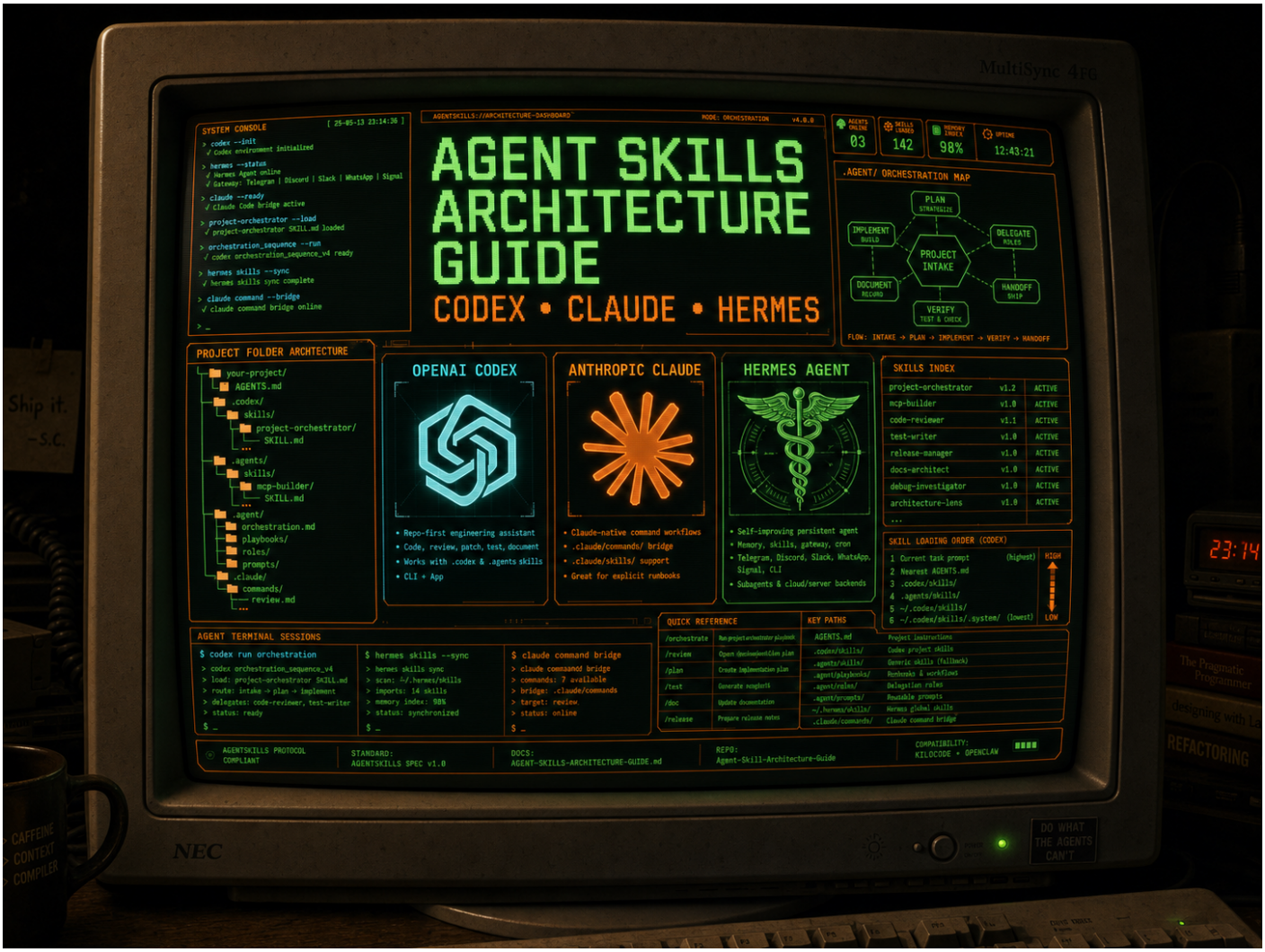


Agent Skills Architecture Guide

Curated by Shane Coy

@shaneswrld_ | github.com/shane9coy

Codex-first orchestration for Codex, Claude, Hermes Agent, KiloCode, and OpenClaw

Agent Skills Architecture Guide

Codex-first folder structure, skill registration, `.agent/` orchestration, Claude command-folder equivalents, and self-hosted model notes

Curated by Shane Coy

@shaneswrld_ | github.com/shane9coy

Updated May 2026

Table of Contents

00 — Codex-First Setup: Instructions, Skills, and Orchestration

The best agent setup is not one giant prompt. It is a small operating system for work:

- `AGENTS.md` defines durable repo instructions.
- `skills/*/SKILL.md` defines reusable capabilities the agent can load when the task matches.
- `.agent/` defines project-local orchestration: roles, playbooks, prompts, review gates, and handoff routes.
- The current chat prompt defines the one task being done right now.

Claude command folders are still a useful mental model: they behave like a CLI menu for workflows the user invokes directly. In a Codex-first setup, treat those commands as either skills or `.agent/playbooks/`:

Need	Codex-first home	Why
Always-on repo rules	`AGENTS.md`	Loaded as durable project guidance
Reusable workflow capability	`skills/<name>/SKILL.md`	Triggered by task type and reusable across runs
Project-specific command/runbook	`.agent/playbooks/<name>.md`	Equivalent to a CLI-style slash command without pretending it is a global skill
Specialist worker/reviewer behavior	`.agent/roles/<role>.md`	Gives subagents or delegated prompts a clear job
Copy/paste task prompt	`.agent/prompts/<name>.md`	Keeps implementation and review prompts consistent

Recommended Codex-first project skeleton:

```
your-project/
|-- AGENTS.md
|-- .agent/
|   |-- README.md
|   |-- orchestration.md
|   |-- roles/
|   |   |-- architect.md
|   |   |-- implementer.md
|   |   |-- reviewer.md
|   |   +-- tester.md
|   |-- playbooks/
|   |   |-- new-feature.md
|   |   |-- bugfix.md
|   |   |-- repo-review.md
|   |   +-- release-handoff.md
|   +-- prompts/
|       |-- implementation-task.md
|       |-- review-task.md
|       +-- handoff-summary.md
|-- skills/
|   +-- project-orchestrator/
|       +-- SKILL.md
```

Use this decision rule:

If the rule is...	Put it in...
Stable for every agent session	`AGENTS.md`
A repeatable capability	`skills/<name>/SKILL.md`
A project route or command menu item	`.agent/playbooks/`
A worker persona or review lane	`.agent/roles/`
A reusable prompt contract	`.agent/prompts/`
Only relevant to this request	The current chat prompt

The goal is to make the agent easier to operate. Do not create ten folders because it looks advanced. Start with one `AGENTS.md`, one orchestrator skill, and two or three playbooks.

01 — How Skills Work Under the Hood

Skills are on-demand prompt expansion — not agents, not plugins, not tools. The framework scans skill folders, reads only the YAML frontmatter (name + description), and builds a compact `<available_skills>` XML list in the system prompt. When a user request matches a description, the full SKILL.md body is injected into context. The model follows the instructions and optionally runs bundled scripts. The core principle is **lazy loading** — skills cost almost nothing until triggered.

The Lifecycle

Phase	What Happens	Context Cost
1. Discovery	Framework scans `*/SKILL.md` in skill paths	0 tokens
2. Indexing	Builds `<available_skills>` from name + description	~24 tok/skill
3. Matching	User request matched against descriptions	0 extra
4. Injection	Full SKILL.md body injected into conversation	Varies (body size)
5. Execution	Model follows instructions, runs scripts	Script output tokens

System prompt overhead is deterministic: ~195 chars base + ~97 chars per skill (plus your name/description lengths). At ~4 chars/token, that's roughly 24 tokens per skill sitting in the system prompt at all times.

Why Skills Over Agents

Skills	Agents
Lightweight — just prompt text	Heavy — full sub-process with own context
Composable — multiple fire in one turn	Isolated — separate conversation thread
Cheap — only loaded when matched	Expensive — own system prompt + tools
Stateless — no memory between calls	Stateful — can maintain own context
Best for: workflows, conventions, tasks	**Best for:** long-running, risky, parallel work

02 — Canonical Folder Structures

Codex is the primary target for this guide. Hermes Agent, KiloCode, and Claude Code can use the same broad AgentSkills pattern, but their folders and runtime behavior are different. Start with Codex unless the project is explicitly being built for another agent.

OpenAI Codex

```
your-project/
|-- AGENTS.md                    # Root project instructions
|-- .agent/                      # Project orchestration map
|   |-- orchestration.md
|   |-- roles/
|   |-- playbooks/
|   +-- prompts/
|-- .codex/                      # Codex project config / native skill location
|   |-- AGENTS.md                # Optional narrower Codex instructions
|   +-- skills/                  # Project skills
|       |-- new-skill/
|       |   +-- SKILL.md
|       +-- project-orchestrator/
|           +-- SKILL.md
|-- .agents/                     # Generic AgentSkills fallback
|   +-- skills/

~/.codex/                        # Global Codex config
|-- AGENTS.md                    # Global instructions
|-- AGENTS.override.md           # Temporary override, higher priority
|-- config.toml                  # CLI/app config and skill enablement
+-- skills/
    |-- .system/                 # Built-in skills
    +-- my-global-skill/
        +-- SKILL.md
```

Codex projects should keep durable rules in root `AGENTS.md`, reusable workflow capability in `.codex/skills/` or `.agents/skills/`, and project-specific orchestration in `.agent/`.

Hermes Agent

Hermes Agent is the Nous Research self-improving agent: https://github.com/NousResearch/hermes-agent. It has a terminal UI, messaging gateway, skills, persistent memory, cron scheduling, subagents, and multiple terminal backends. It can migrate from OpenClaw using `hermes claw migrate`.

```
~/.hermes/                      # User-level Hermes home
|-- skills/                     # User-created and imported skills
|   +-- openclaw-imports/       # Imported OpenClaw skills, when migrated
|-- memory/                     # Persistent memory and user profile material
|-- config.*                    # Provider, tools, gateway, and runtime config
+-- sessions/                   # Conversation/session state

hermes-agent repo/runtime:
|-- AGENTS.md                   # Repo instructions for Hermes development
|-- skills/                     # Built-in or repo-level skills
|-- optional-skills/            # Optional skill packs
|-- cron/                       # Scheduled automation support
|-- gateway/                    # Messaging gateway
|-- hermes_cli/                 # CLI/TUI entrypoint
+-- tools/                      # Tool and toolset implementation
```

Use Hermes when the target workflow needs persistent memory, messaging platforms, built-in cron, autonomous skill improvement, cross-session recall, or cloud/server terminal backends. Use Codex when the target workflow is primarily local repo engineering and code review inside the Codex CLI/app loop.

KiloCode

```
your-project/
|-- .kilocode/
|   |-- AGENTS.md               # KiloCode project memory
|   |-- skills/                 # Generic skills, all modes
|   |   +-- new-skill/
|   |       +-- SKILL.md
|   |-- skills-code/            # Code mode only
|   |   +-- linting-rules/
|   |       +-- SKILL.md
|   +-- skills-architect/       # Architect mode only
|       +-- microservices/
|           +-- SKILL.md

~/.kilocode/
|-- AGENTS.md
|-- skills/
+-- skills-code/
```

KiloCode's distinctive feature is mode-scoped skills: `skills-code/`, `skills-architect/`, `skills-debug/`, etc.

Claude Code

```
your-project/
|-- .claude/
|   |-- CLAUDE.md               # Project memory
|   |-- settings.json           # Permissions, env vars
|   |-- commands/               # Custom slash commands
|   |   +-- review.md           # Invoked via /review
|   +-- skills/
|       |-- mcp-builder/
|       |   |-- SKILL.md
|       |   |-- scripts/
|       |   |-- references/
|       |   +-- assets/
|       +-- new-skill/
|           +-- SKILL.md

~/.claude/
|-- CLAUDE.md
+-- skills/
```

Claude command folders are useful as a mental model for explicit, user-triggered runbooks. In this guide, the Codex-first equivalent is `.agent/playbooks/` plus reusable skills.

Cross-Platform Path Reference

	OpenAI Codex	Hermes Agent	KiloCode	Claude Code
Project instructions	`AGENTS.md`	`AGENTS.md` / context files	`AGENTS.md`	`CLAUDE.md`
Project skills	`.codex/skills/ .agents/skills/`	project/repo `skills/` when configured	`.kilocode/skills/`	`.claude/skills/`
Global skills	`~/.codex/skills/`	`~/.hermes/skills/`	`~/.kilocode/skills/`	`~/.claude/skills/`
Orchestration folder	`.agent/` playbooks/roles/prompts	Hermes gateway, memory, cron, subagents	modes + skills	commands + agents
Config	`~/.codex/config.toml`	Hermes config / `hermes config set`	VS Code settings	`.claude/settings.json`
Messaging gateway	No native gateway	Yes: Telegram, Discord, Slack, WhatsApp, Signal, CLI	No	No
Cron / schedules	External automation	Built-in cron scheduler	No	No
Mode-scoped skills	No	No	Yes: `skills-{mode}/`	No
OpenClaw migration	N/A	`hermes claw migrate`	N/A	N/A

Key Differences

- **Codex is repo-first.** Use root `AGENTS.md`, project skills, and `.agent/` playbooks to make local engineering workflows repeatable.
- **Hermes Agent is persistent-agent-first.** It adds memory, a messaging gateway, cron scheduling, cloud/server terminal backends, autonomous skill creation, and cross-session search.
- **KiloCode is mode-first.** Use mode-specific skill folders when the same repo needs different behavior in code, architecture, ask, or debug modes.
- **Claude Code is command-friendly.** Its `.claude/commands/` pattern maps well to Codex `.agent/playbooks/` when you want explicit user-triggered workflows.

  Critical (all platforms): *Each skill MUST be in its own named subfolder.* `skills/SKILL.md` *will NOT register — it must be* `skills/my-name/SKILL.md`*.*

  For maximum Codex compatibility, *ship repo instructions in* `AGENTS.md`*, Codex skills in* `.codex/skills/` *or* `.agents/skills/`*, and project runbooks in* `.agent/playbooks/`*.*

03 — SKILL.md Anatomy & Frontmatter

```
---
name: my-skill-name
description: "Concise trigger. Use when [X]. Handles [Y, Z]."
---

# Skill Title

## Instructions
Step-by-step instructions the model follows...

## Examples
Input/output pairs showing expected behavior...

## Error Handling
What to do when things fail...
```

Frontmatter Reference

Field	Req	Purpose
`name`	YES	Unique ID. Max 64 chars. Becomes /skill-name command.
`description`	YES	**THE trigger.** Max 200 chars. Must include WHAT + WHEN.
`license`	No	License info string
`context`	No	`fork` = subagent, `inline` = current context (default)
`disable-model-invocation`	No	`true` = only user can invoke (for destructive ops)
`user-invocable`	No	`false` = only model can invoke (background knowledge)
`allowed-tools`	No	Restrict tools: Read, Grep, Glob, Bash, Write
`model`	No	Override model for this skill only
`metadata`	No	Single-line JSON. Extra key-value data.

Description Do's and Don'ts

Example	Verdict
`Helps with development tasks`	■ BAD
`This handles [brackets] and #tags`	■ BAD
`"Create MCP servers. Use when building API integrations."`	■ GOOD
`"Deploy to production. Use when pushing releases or shipping."`	■ GOOD

> ***Most common failure:*** *Unquoted descriptions with special YAML characters cause the skill to be silently skipped. Always wrap descriptions in double quotes.*

04 — Skill Subdirectory Conventions

Directory	Contents	How the Model Uses It
`scripts/`	Python, Bash, JS executables	Runs via Bash tool. Ref: `{baseDir}/scripts/x.py`
`references/`	API docs, schemas, guides	Read on-demand via progressive disclosure
`assets/`	Templates, configs, images	Copied or modified during output generation

Progressive Disclosure

Show the model just enough to decide, then reveal more as needed. This keeps context lean and costs low.

Layer	What's Exposed	When
1. Frontmatter	name + description (system prompt)	Always — every request
2. SKILL.md body	Full instructions + examples	When skill is triggered
3. references/*	Deep docs, schemas, API refs	When SKILL.md says to read them
4. scripts/*	Executable code + output	When instructions call for execution

> *Keep SKILL.md under 500 lines. In the body, write: "For details, read* `{baseDir}/references/api-docs.md`"

05 — Precedence & Loading Order

Precedence is the order the agent should use when multiple instruction or skill sources exist. Codex comes first in this guide because this is now a Codex-first reference.

OpenAI Codex

Priority	Location	Scope
1 (highest)	Current task prompt	This request only
2	Nearest applicable `AGENTS.md` / `AGENTS.override.md`	Repo or directory-specific instructions
3	`<project>/.codex/skills/`	Codex skills for this project
4	`<project>/.agents/skills/`	Generic AgentSkills fallback
5	`~/.codex/skills/`	All Codex projects for this user
6 (lowest)	`~/.codex/skills/.system/`	Built-in defaults

Codex projects should keep stable rules in `AGENTS.md`, workflow capability in skills, and project command/runbook behavior in `.agent/playbooks/`.

Hermes Agent

Priority	Location	Scope
1 (highest)	Current conversation / command	This request only
2	Active context files / repo `AGENTS.md`	Project and session instructions
3	Project or repo `skills/`	Project skills, when configured
4	`~/.hermes/skills/`	User-created and imported skills
5	Hermes built-in / optional skills	Runtime defaults and optional packs
6 (lowest)	Imported OpenClaw skills under `~/.hermes/skills/openclaw-imports/`	Migration support, review before relying on them

Hermes is strongest when you want skills plus memory, messaging, schedules, and a persistent agent runtime. If you are migrating from OpenClaw, use `hermes claw migrate` and then audit the imported skills before treating them as current source of truth.

KiloCode

Priority	Location	Scope
1 (highest)	`<project>/.kilocode/skills-{mode}/`	This project, this mode only
2	`<project>/.kilocode/skills/`	This project, all modes
3	`~/.kilocode/skills-{mode}/`	Global, mode-specific
4	`~/.kilocode/skills/`	Global, all modes
5 (lowest)	`.claude/skills/` (fallback)	Cross-tool compatibility

> ***KiloCode's unique feature:*** *mode-specific skill directories. Use* `skills-code/` *for coding mode,* `skills-architect/` *for architecture mode,* `skills-debug/` *for debug mode, etc. Generic* `skills/` *applies to all modes. The directory pattern is* `skills-{mode-slug}` *where mode-slug matches the agent mode identifier.*

Claude Code

Priority	Location	Scope
1 (highest)	`<project>/.claude/skills/`	This project only
2	`~/.claude/skills/`	All projects for this user
3 (lowest)	Marketplace / bundled plugins	Global defaults

Claude remains useful as a comparison point, especially for command folders, but Codex should be the first implementation target in this guide.

06 — AGENTS.md vs SKILL.md vs `.agent/` vs Commands

Codex-first projects need four separate instruction layers. Mixing them is what makes agent setups feel unpredictable.

Layer	Loaded	Purpose	Best For
`AGENTS.md`	Every session / repo context	Persistent project instructions	Stack, conventions, architecture rules, testing bar
`skills/*/SKILL.md`	On-demand	Reusable capability	Debug workflows, review lanes, MCP builders, release prep
`.agent/playbooks/*.md`	When referenced by user or orchestrator	CLI-style project runbooks	`/review`, `/bugfix`, `/ship`, `/build-agent` equivalents
`.agent/roles/*.md`	When delegating or spawning	Specialized worker behavior	Architect, implementer, reviewer, tester
`.agent/prompts/*.md`	When copied into a task	Prompt contracts	Implementation tasks, review tasks, handoff summaries
Tool config	Session start	Permissions and runtime config	Env vars, model routing, tool access

Claude Commands vs Codex Playbooks

Claude command folders are user-triggered runbooks. The user types a command and the command file becomes the task context. Codex does not need that exact folder model to get the same benefit.

In Codex, use `.agent/playbooks/` as the CLI-style layer:

```
.agent/
  playbooks/
    new-feature.md
    bugfix.md
    repo-review.md
    release-handoff.md
```

Then tell Codex:

```
Use .agent/playbooks/bugfix.md for this bug.
```

Or use an orchestration skill that routes automatically:

```
Use the project-orchestrator skill. This is a bugfix with test coverage and review.
```

Decision Flowchart

```
Does the agent need this rule every time?
    |
    YES --> AGENTS.md
    |
    NO
    |
    Is this a reusable capability across tasks?
        |
        YES --> skills/my-skill/SKILL.md
        |
        NO
        |
        Is this a project-specific command or route?
            |
            YES --> .agent/playbooks/action.md
            |
            NO
            |
            Is this a specialist worker/reviewer role?
                |
                YES --> .agent/roles/worker.md
                |
                NO
                |
                Is this a reusable prompt shape?
                    |
                    YES --> .agent/prompts/task.md
                    |
                    NO  --> Current chat prompt
```

07 — Codex vs Hermes Agent vs KiloCode vs Claude Code

All four can use `SKILL.md`-style procedural knowledge, but they optimize for different workflows. For this guide, treat Codex as the default local engineering agent and Hermes Agent as the persistent, self-improving agent runtime.

Folder Structure Comparison

CODEX

```
project/
|-- AGENTS.md
|-- .agent/
|   |-- orchestration.md
|   |-- roles/
|   |-- playbooks/
|   +-- prompts/
|-- .codex/
|   +-- skills/
|       +-- review/
|           +-- SKILL.md
|-- .agents/       # Generic fallback
|   +-- skills/
~/.codex/
|-- AGENTS.md
|-- config.toml
+-- skills/ (global)
```

HERMES AGENT

```
~/.hermes/
|-- skills/
|   +-- openclaw-imports/        # Optional migration output
|-- memory/
|-- sessions/
+-- config.*

hermes-agent runtime:
|-- AGENTS.md
|-- skills/
|-- optional-skills/
|-- cron/
|-- gateway/
|-- hermes_cli/
+-- tools/
```

KILOCODE

```
project/
|-- .kilocode/
|   |-- AGENTS.md
|   |-- skills/
|   |   +-- review/
|   |       +-- SKILL.md
|   +-- skills-code/
|       +-- lint/
|           +-- SKILL.md
~/.kilocode/
    +-- skills/ (global)
```

CLAUDE CODE

```
project/
|-- .claude/
|   |-- CLAUDE.md
|   |-- settings.json
|   |-- commands/
|   +-- skills/
|       +-- review/
|           +-- SKILL.md
~/.claude/
    +-- skills/ (shared)
```

Feature Comparison

Feature	Codex	Hermes Agent	KiloCode	Claude Code
Best fit	Local repo engineering, code review, implementation	Persistent personal/team agent with memory and messaging	Mode-scoped IDE workflows	Claude-native command and skill workflows
Project instructions	`AGENTS.md`	`AGENTS.md` / context files	`AGENTS.md`	`CLAUDE.md`
Project skills	`.codex/skills/ .agents/skills/`	repo/project `skills/` when configured	`.kilocode/skills/`	`.claude/skills/`
Global skills	`~/.codex/skills/`	`~/.hermes/skills/`	`~/.kilocode/skills/`	`~/.claude/skills/`
Runbook layer	`.agent/playbooks/`	slash commands + skills + cron	modes + skills	`.claude/commands/`
Config	`config.toml`	`hermes config set` / config files	VS Code settings	`settings.json`
Memory	Repo instructions and conversation context	Persistent memory, user modeling, session search	Project memory	Project memory
Messaging	CLI + app	CLI plus Telegram, Discord, Slack, WhatsApp, Signal	VS Code + CLI	CLI + IDE
Cron	External automation	Built-in scheduler	No	No
Subagents	Delegated workers where supported	Isolated subagents and RPC tools	Mode/workflow dependent	Agent/subagent patterns where supported
Self-hosted models	Provider-dependent	Broad provider/model support, plus custom endpoints	Provider/key dependent	Provider/base-url dependent
OpenClaw migration	N/A	`hermes claw migrate`	N/A	N/A

Practical Selection Rule

Use **Codex** when the work is a local repo change, code review, architecture patch, test run, or documentation update.

Use **Hermes Agent** when the workflow needs persistent memory, messaging delivery, scheduled work, autonomous skill improvement, or a cloud/server agent that is not tied to your laptop.

Use **KiloCode** when mode-specific skill behavior matters inside the KiloCode environment.

Use **Claude Code** when the project is already centered on `.claude/commands/`, `.claude/skills/`, or Claude-native workflows.

08 — Codex Orchestration: Agent Folders to Runtime

How folder structure translates to runtime behavior. Understanding this mapping is key to structuring your project correctly.

The practical Codex pattern is:

1. `AGENTS.md` sets the operating rules.
2. `skills/` supplies reusable capabilities.
3. `.agent/` supplies the project-specific orchestration map.
4. The current Codex thread acts as the controller unless you deliberately delegate work to subagents.

Codex Runtime Flow

```
USER REQUEST
    |
    v
+--------------------+
| CODEX CONTROLLER  |  <-- Reads AGENTS.md / repo instructions
| (current thread)  |  <-- Sees available skills
+---------+---------+
    |
    +--> Check .agent/orchestration.md when project routing matters
    |
    +--> Select playbook from .agent/playbooks/
    |
    +--> Load matching skill body when the task matches a skill
    |
    +--> Delegate only when work can be bounded by role + file ownership
    |
    v
IMPLEMENT --> REVIEW --> VERIFY --> HANDOFF
```

`.agent/` Orchestration Folder

A `.agent/` folder is a project convention, not magic by itself. Its job is to make orchestration visible and repeatable.

```
.agent/
|-- README.md
|-- orchestration.md
|-- roles/
|   |-- architect.md
|   |-- implementer.md
|   |-- reviewer.md
|   +-- tester.md
|-- playbooks/
|   |-- new-feature.md
|   |-- bugfix.md
|   |-- repo-review.md
|   +-- release-handoff.md
+-- prompts/
    |-- implementation-task.md
    |-- review-task.md
    +-- handoff-summary.md
```

Recommended file purposes:

File / Folder	Purpose
`.agent/README.md`	Explains the folder contract for humans and agents.
`.agent/orchestration.md`	Defines the default route from intake to handoff.
`.agent/roles/`	Defines worker and reviewer responsibilities.
`.agent/playbooks/`	Stores command-like workflows for this project.
`.agent/prompts/`	Stores reusable prompt contracts for implementation, review, and handoff.

Orchestration Agent Responsibilities

An orchestration agent is the controller. It should not blindly write all the code or all the documentation itself.

Its responsibilities:

1. Read `AGENTS.md` and the task.
2. Identify the stack, service boundaries, data layer, integrations, deployment shape, and operational constraints.
3. Choose the right skill or `.agent/playbook`.
4. Split broad work into small tasks with clear file ownership.
5. Dispatch or simulate roles: architect, implementer, reviewer, tester.
6. Run review gates before declaring the work complete.
7. Return a handoff with files changed, commands run, open risks, and next steps.

The controller should preserve Shane's role as product owner and technical operator. It can recommend a route, but it should not invent product decisions, credentials, production data, or external state.

Example Codex Orchestrator Skill

```
---
name: project-orchestrator
description: "Use when a task needs stack review, architecture, skill routing,
implementation slices, review gates, validation, and handoff."
---

# Project Orchestrator

## Workflow
1. Read AGENTS.md.
2. Read .agent/orchestration.md.
3. Classify the task: feature, bugfix, review, release, docs, or setup.
4. Select the matching .agent/playbook.
5. Load any matching skills.
6. Break work into bounded tasks.
7. Implement, review, verify, and hand off.
```

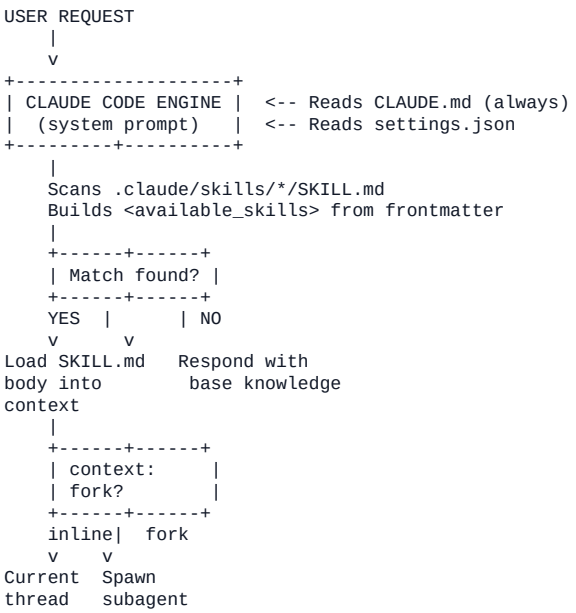
Use this skill when the request is more than a one-file edit. It keeps the agent from collapsing architecture, implementation, testing, and delivery into one unreviewed pass.

Hermes Agent Runtime Flow

```
MESSAGE (CLI / Telegram / Discord / Slack / WhatsApp / Signal)
    |
    v
+--------------------+
| HERMES INTERFACE  |  <-- `hermes` TUI or `hermes gateway`
| CLI / gateway     |  <-- shared slash commands where supported
+---------+---------+
    |
    Loads config, tools, memory, session context, and skills
    |
    +------+------+
    | Model Call   |  <-- Nous Portal / OpenRouter / OpenAI / local endpoint / other
provider
    +------+------+
    |
    Skill matched        --> SKILL.md procedure used
    Memory relevant      --> persistent memory/session search used
    Cron scheduled       --> runs unattended with platform delivery
    Subagent delegated   --> isolated workstream or RPC tool pipeline
    |
    v
Response --> CLI or messaging platform
```

Hermes differs from the old OpenClaw framing in this guide because it is not just a workspace gateway. It is a self-improving agent runtime with memory, skills, cron, messaging, subagents, and migration support for OpenClaw users.

Claude Code Runtime Flow



Folder-to-Runtime Mapping

Folder / File	Loaded At	Runtime Effect
`AGENTS.md`	Session start / repo context	Codex-first project instructions. Always present when applicable.
`.agent/orchestration.md`	When project routing matters	Defines route from intake to handoff.
`.agent/playbooks/*.md`	When referenced by user or orchestrator	Command-like workflow for this project.
`.agent/roles/*.md`	When delegating or spawning	Defines specialist worker/reviewer behavior.
`.agent/prompts/*.md`	When copied into a task	Reusable prompt contract.
`.codex/skills/x/SKILL.md`	On skill discovery / trigger	Codex project capability.
`.agents/skills/x/SKILL.md`	On skill discovery / trigger	Generic AgentSkills fallback.
`~/.codex/skills/x/SKILL.md`	On skill discovery / trigger	User-wide Codex capability.
`~/.hermes/skills/x/SKILL.md`	Hermes skill discovery / trigger	User-wide Hermes capability.
`~/.hermes/memory/`	Hermes session/runtime	Persistent memory and user profile material.
Hermes cron config	Hermes runtime	Scheduled unattended tasks with delivery to platforms.
`.claude/commands/x.md`	User types `/x`	Claude command/runbook; Codex equivalent is `.agent/playbooks/x.md`.
`.claude/skills/x/SKILL.md`	Claude skill trigger	Claude-specific skill location.

Multi-Agent Orchestration (Codex + Hermes)

```
+----------------------+
|  CODEX CONTROLLER    |
|  repo-local work     |
+----------+-----------+
           |
           +--> .agent/playbooks/ define route
           +--> skills/ define reusable capabilities
           +--> roles/ bound delegated workers
           |
           v
  IMPLEMENT --> REVIEW --> VERIFY --> HANDOFF

+----------------------+
|  HERMES AGENT        |
|  persistent runtime  |
+----------+-----------+
           |
           +--> memory recalls cross-session context
           +--> gateway delivers through messaging apps
           +--> cron schedules recurring work
           +--> subagents parallelize isolated workstreams
           |
           v
  PLAN --> ACT --> LEARN --> REMEMBER --> SCHEDULE
```

09 — Self-Hosted Model Configuration

Skills are framework-level procedures. They can work with Codex, Hermes Agent, KiloCode, Claude Code, or another AgentSkills-compatible runtime as long as the runtime can discover the skill, load the instructions, and call the required tools.

Codex Provider Posture

For Codex, keep model/provider settings in Codex configuration and keep workflow behavior in `AGENTS.md`, `.agent/`, and skills. Do not bake secrets or provider-specific API keys into skills.

```
~/.codex/
|-- config.toml
|-- AGENTS.md
+-- skills/
```

Use Codex for the local repo workflow. Use Hermes Agent when you need a persistent gateway, cron, messaging delivery, or a long-lived cloud/server runtime.

Hermes Agent Provider Posture

Hermes supports many providers and custom endpoints through its setup and model commands. Use the CLI instead of hardcoding provider blocks in a skill:

```
hermes setup          # full setup wizard
hermes model          # choose provider and model
hermes config set     # set individual config values
hermes doctor         # diagnose config/runtime issues
```

Hermes is a better fit than plain local config when the agent needs memory, scheduled work, messaging platforms, or server/cloud terminal backends.

Ollama (local endpoint)

```
# Install + pull a model
curl -fsSL https://ollama.ai/install.sh | sh
ollama pull qwen3:8b
```

Use Ollama as a local OpenAI-compatible endpoint only after confirming the target runtime supports that routing path.

LM Studio (GUI, good for experimenting)

```
Base URL: http://127.0.0.1:1234/v1
API key: lm-studio or any placeholder accepted by your local server
Model: the model loaded in LM Studio
```

vLLM (production-style local serving)

```
# Install and serve
pip install vllm
vllm serve meta-llama/Llama-3.1-70B-Instruct \
  --served-model-name llama-3.1-70b \
  --api-key YOUR_KEY \
  --port 8000 \
  --enable-auto-tool-choice \
  --max-model-len 131072
```

Claude Code + Local Models

Claude Code can route through compatible endpoints when configured for that environment. Keep this as a Claude-specific fallback, not the default path for this guide.

```
export ANTHROPIC_BASE_URL=http://localhost:11434
# Then run Claude Code with the configured endpoint.
```

Model Recommendations by VRAM

VRAM	Model	Quant	Notes
< 2 GB	Qwen3 1.7B Thinking	Q4_K_M	Ultra-light, runs on anything
2-4 GB	Qwen3 4B Thinking	Q4_K_M	Outperforms many 8B models at tool calling
4-8 GB	Qwen3 8B / Qwen3 4B	Q4_K_M	Sweet spot for older GPUs
8-16 GB	Qwen3 8B	Q6_K / FP16	Best all-around for most laptops
24 GB	GLM 4.7 Flash / GPT-OSS 20B	Q4_K_M	Strong coding + tool use
48 GB	GLM 4.7 Flash (30B)	Q4_K_M	Stronger local coding and reasoning
48-80 GB	Qwen3 Coder (80B)	Q4_K_XL	Near-API quality when served well
90+ GB	GPT-OSS 120B	Q4_K_M	Maximum local capability

> *Set context windows high enough for the target workflow. Agentic coding runs often need repo instructions, skill bodies, tool results, and test output in the same session.*

Security Warnings

Self-hosted agents execute commands on your machine. Take precautions:

Risk	Mitigation
Command injection	Run risky workloads in Docker, sandboxed terminals, or remote throwaway environments
Network exposure	Never expose local model or gateway ports publicly without auth
Workspace escape	Use container isolation, least-privilege mounts, and explicit working directories
Malicious skills	Audit third-party skills before installing

Risk	Mitigation
Data exfiltration	Do not put secrets into skills, prompts, logs, or copied examples
Persistent-agent drift	In Hermes, periodically review memory, cron jobs, skills, and imported migrations

10 — Troubleshooting: Skills Not Registering

Symptom	Cause	Fix
Skill not in list	Flat file: `skills/SKILL.md`	Must be `skills/<n>/SKILL.md`
Silently skipped	Unquoted description	Wrap in double quotes
Never triggers	Vague description	Add specific trigger keywords
Frontmatter ignored	Missing `---` markers	Must start and end with `---`
1 of N loads	YAML special chars in others	Quote ALL descriptions
Triggers but fails	Missing referenced files	Check `scripts/`, `references/`, and `assets/` exist
Too much context	SKILL.md > 500 lines	Move depth to `references/`
Wrong agent folder	Skill copied to the wrong runtime path	Use `.codex/skills/`, `.agents/skills/`, or `~/.codex/skills/` for Codex
Imported migration drift	Old OpenClaw skill imported into Hermes without review	Audit `~/.hermes/skills/openclaw-imports/` before use
Model ignores skill	Trigger info in body only	Move “when to use” language into `description`

Diagnostic Commands

```
# Codex-first: find project skills
find .codex/skills .agents/skills skills -name "SKILL.md" 2>/dev/null

# Dump frontmatter
for f in $(find .codex/skills .agents/skills skills -name "SKILL.md" 2>/dev/null); do
  echo "=== $f ==="
  sed -n '/^---$/,/^---$/p' "$f"
done

# Find unquoted descriptions
grep -R -n "^description:" .codex/skills .agents/skills skills 2>/dev/null | grep -v '"'

# Check sizes
find .codex/skills .agents/skills skills -name "SKILL.md" -print0 2>/dev/null | xargs -0 wc
-l | sort -rn | head

# Hermes: inspect setup and skills
hermes doctor
hermes skills 2>/dev/null || true
```

11 — Installing External Skills

Codex-First Install From GitHub

```
git clone --depth 1 <repo-url> /tmp/skill-clone
SKILL=$(find /tmp/skill-clone -name "SKILL.md" | head -1)
NAME=$(basename "$(dirname "$SKILL")")
mkdir -p .codex/skills/$NAME
cp -r "$(dirname "$SKILL")"/* .codex/skills/$NAME/
rm -rf /tmp/skill-clone
```

Install Into Generic `.agents/` Fallback

```
git clone --depth 1 <repo-url> /tmp/skill-clone
SKILL=$(find /tmp/skill-clone -name "SKILL.md" | head -1)
NAME=$(basename "$(dirname "$SKILL")")
mkdir -p .agents/skills/$NAME
cp -r "$(dirname "$SKILL")"/* .agents/skills/$NAME/
rm -rf /tmp/skill-clone
```

From Zip

```
unzip skill.zip -d /tmp/install
SKILL=$(find /tmp/install -name "SKILL.md" | head -1)
NAME=$(grep "^name:" "$SKILL" | sed 's/name: *//' | tr -d '"')
mkdir -p .codex/skills/$NAME
cp -r "$(dirname "$SKILL")"/* .codex/skills/$NAME/
```

Post-Install Validation

```
ls -la .codex/skills/<n>/
head -5 .codex/skills/<n>/SKILL.md
# Then: ask Codex something matching the skill description.
```

12 — Quick Skill Template

```
---
name: your-skill-name
description: "What it does. Use when [trigger]. Handles [X, Y]."
---

# Your Skill Name

## Overview
One paragraph explaining purpose.

## Instructions
1. First step
2. Second step
3. Third step

## Examples

### Example 1: [Scenario]
**Input:** "user says this"
**Action:** Do X, then Y
**Output:** Expected result

## Error Handling
- If X fails, do Y

## Notes
- Keep under 500 lines
- Move depth to references/
```

Appendix A — The new-skill Installer Skill

Install the new-skill SKILL.md in your Codex skill folder to give your CLI agent the ability to install, scaffold, validate, and manage other skills.

```
mkdir -p .codex/skills/new-skill
cp new-skill-SKILL.md .codex/skills/new-skill/SKILL.md

# For cross-agent compatibility, you can also mirror it:
mkdir -p .agents/skills/new-skill
cp new-skill-SKILL.md .agents/skills/new-skill/SKILL.md
```

Built-in Workflows

#	Workflow	Trigger Example
1	Create from scratch	"create a new skill for X"
2	Install from URL/repo/zip	"install this skill: <url>"
3	Install MCP Builder	"set up the mcp-builder skill"
4	Validate a skill	"check if my skill is correct"
5	Audit agent folder	"scan and fix my .codex/.agents structure"

Appendix B — AGENTS.md Starter Template

```
# Project Configuration

Shane is the product owner, technical operator, and admin.

Act as a senior software engineer. Start stack-first and architecture-first before
substantial implementation.

## Stack
- Language: [Python 3.12 / TypeScript 5.x]
- Framework: [FastAPI / Next.js]
- Database: [PostgreSQL / SQLite]
- Deployment: [Docker / Vercel]

## Commands
- `npm run dev`    -- Start dev server
- `npm test`       -- Run test suite
- `npm run build`  -- Production build
- `npm run lint`   -- Lint and format

## Code Conventions
- TypeScript strict mode
- Functional components with hooks
- Tailwind for styling
- Tests colocated next to source

## Architecture
- /src/components  -- React components
- /src/lib         -- Shared utilities
- /src/api         -- Route handlers
- /src/types       -- Type definitions

## Notes
- Run tests before committing
- Conventional commits (feat:, fix:)
- Never commit .env files
- Do not invent production data, credentials, policies, or external state
```

Agent Skills Architecture Guide

Curated by Shane Coy

@shaneswrld_ | github.com/shane9coy

OpenAI Codex / Hermes Agent / KiloCode / Claude Code / Ollama / vLLM

v4.2 — May 2026